FRED TALKS

30th October 2026

CONTENTS

The lies I used to tell myself

CATE HALL · 1900 WORDS

Ownership

THORSTEN BALL · 1122 WORDS

Zen of AI Coding

YOAV AVIRAM · 1912 WORDS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 1053 WORDS

The lies I used to tell myself

CATE HALL · 23 JAN 2026 · SOURCE



1. What doesn’t kill you makes you stronger.

When I was 31, my husband of four years, who I loved more than my own life, left me completely out of the blue. We’d had an incredible, epic romance — drawn together like magnets, we married on our third date and spent the years that followed burrowing ever closer together. Maybe that was the problem — I can’t say for sure, because I never got a satisfying explanation out of him. One day I was the protagonist of a Princess Bride-worthy love story; 48 hours later, he moved out.

From the time we met, I had never contemplated the possibility of living without him, and I didn’t want to. I wanted to die for a long time — at least a couple of years. But as I slowly learned to sequester away thoughts of him and the emotions they brought up, I became convinced of a silver lining: The experience would make me impenetrable. If I survived it, I would know I could survive anything, and nothing would be able to hurt me again.

It took the better part of a decade and another deeply identity-shifting relationship, with the man who became my second husband, for me to see that I had it all wrong: Impenetrability means avoiding the parts of life that are most life-like. There’s a reason the visual metaphor for love is an arrow.

The near-fatal catastrophes of our lives don’t always make us stronger. Surviving one hard thing does provide evidence that you can survive other hard things, which is easy to *mistake* for getting stronger. But in reality they often just make us brittle, make us dissociate and flee from the situations that remind us we can be broken.

2. When you know, you know.

Mario Gabriele of the Generalist had a great "50 Things" style post a few months ago that I found myself nodding along to almost the whole way through. The glaring exception was Number 33: "Love is easy. The difference between bad relationships and meeting your spouse is not a matter of degree, but category. Compatibility is unignorable and effortless."

I believed this completely, once. I've since come to believe that what's unignorable and effortless is chemistry. Chemistry can be powerful enough to turn your life sideways, but it's not the same thing as compatibility. Compatibility is litigated on the timescale of years and decades; it's not a momentary feeling but a description of what happens during an extended course of change. Do you grow toward one another, or do you grow apart? Does the relationship help you move closer to the person you aspire to be?

I "just knew" with my first husband. I wasn't sure with my second. Four years in, the signs have flipped.

3. If it were a good idea, someone would be doing it already.

There's a belief, especially common among wonky types, that you don't find $20 bills on the sidewalk — if an opportunity were real, someone would have seized it already. This sounds worldly and wise, but it's actually a form of defensive thinking. It assumes the current way is best, that everything worth doing is being done.

In my experience, this is just false. When I started playing poker, I discovered that physical tells were basically a wide-open field — despite the fact that most pros believed the topic was played out. If you think you see an opportunity others have missed, you should have a story for why they're missing it, and you should ask around to stress-test that story. But if no one can give you a satisfying reason, don't assume one exists. Sometimes the answer really is just inertia or fear.

4. If it's worth doing, it's worth doing well.

Almost nothing is worth doing "well." Many things can and should be done to a minimal standard of quality, because anything more is a waste of effort that can be better allocated. The exceptions aren't things that should be done "well," but things that should be done to an exceptionally high standard, probably a much higher one than you see modeled by the people around you.

This is not semantics; most people really "do everything the way they do anything" — whether that's in a low-effort or high-effort way. But the point of phoning it in on a lot of stuff is to save your energy to expend on the 1 in 1000 things that really matter … and then to actually spend it.

A recent example: I decided that I want to narrate my own audiobook for *You Can Just Do Things*. When I told a friend in the industry this and asked whether they knew any good coaches to work with, they said that hiring a coach wasn't normal or necessary, because my publisher would decide based on a sample whether I was good enough at reading to do it, and if so give me a producer to work with day-of. But it "1 in 1000" matters to me whether the audiobook is so good no one would ever think of returning it on account of the narration, so I'm not interested in the normal-shaped solution that constitutes doing it "well," I'm interested in the one that causes me to come to mind when someone asks this question. In this case, that means hiring multiple coaches and practicing for months.

5. If you like your job, you're doing something wrong.

Back when I was more alienated from my emotions, I thought about work like this: You have a budget of energy to spend, and the goal is to efficiently convert that energy into impact. You identify the activity that generates the highest impact per unit of energy, and then you spam that button for as many waking hours as you can stomach. Is it any wonder I was always burning out?

What I failed to understand is that energy is not fixed. It's dramatically affected by what you choose to do — some activities increase your energy over time, others deplete it. This feels so obvious to me now that I feel kind of stupid including it here, but I know many people (they are endemic to the Bay Area) who still labor under the false belief. Whether you enjoy what you're doing affects your ability to do it year in and year out, and dismissing this as "selfish" doesn't make it untrue. As my husband puts it, the best use of effortful motivation is to engineer your life to require less of it.

6. "All people are insane. They will do anything at any time, and God help anybody who looks for reasons."

This line, from Kurt Vonnegut, used to neatly encapsulate the way I understood other people — which is to say, not at all. Trying to figure out why people did what they did, or imagine what they'd do next, seemed like a fool's errand. I was judgmental about it, too: I assumed this was a problem, not with me, but with everyone else.

I didn't fundamentally move beyond this belief until recent years, when I came into contact with the Enneagram. I imagine there are other personality typing systems that can do something similar, but the Enneagram was magical for me because it clued me into the fact that people can have very different motivational systems from my own that are still internally coherent and produce good things in the world.

People's behavior was previously incomprehensible to me because I was imputing my own motivational system to them, trying to figure out what would cause me to act the way they did — essentially, concluding that *their* actions made no sense in light of *my* goals. But people have wildly diverse emotional hardware, which determines what's rational for them. And my particular personality — independent, moralistic, systematic, challenge-seeking, competitive — is unusual, so I was especially poorly placed to measure others by the yardstick of myself.

This shift in perspective was dramatically good for me and for my relationships with other people. It let me see the ecosystem of psychologies as a kind of miracle — wow, it really does take all kinds — rather than a prompt for cosmic horror. And it gave me confidence that, if I invested in understanding people more deeply, they would eventually cease to be total mysteries, prone to bizarre and unreasonable behavior. As far as I can tell, this is the better part of emotional intelligence.

7. Money can't buy happiness.

The good version of this advice is: You'll still have to work at being happy, even if you're rich. Money in your bank account won't do it for you, and wealth comes with its own pathologies, like endless status-chasing. But the common gloss — that money stops mattering past some modest threshold — is basically false.

You may have heard there's research showing money doesn't buy happiness. That research was explosively popular precisely because it was counterintuitive — it betrayed the commonsense intuition that more money means more freedom, more options, more ability to help people you care about. Turns out it was also riddled with methodological problems. More recent work by Matthew Killingsworth (nominative determinism FTW), using large-scale studies that pinged participants about their happiness at random moments, found steady returns to well-being with rising income, no plateau in sight. The myth persists because it serves a psychological function: It helps people without money feel better about their situation, and helps the wealthy downplay their advantages. But it's not true.

8. Intuition is fake and rationality solves everything.

I used to be horribly judgmental of people who made decisions based on intuition, as in, "I just do what feels right." And then I noticed that whenever I overrode my intuitions, I made *terrible* decisions. I hired the wrong people, started projects with the wrong people, dated the wrong people, ate food that made me feel bad, did forms of exercise I found unfun and injurious. I don't have a satisfying theory for why this is, but I know what happens: When I shift from emotional intuition to a logical story about a decision, it's much easier for my reasoning to get hijacked by plausible-sounding directives that turn out to be nonsense.

It took me a long time to learn this, and that might have something to do with my training in law and poker. Overriding intuition in favor of explicit reasoning makes sense in contexts that reward systematic rationality, where you will get separated from your money if you count on gut feelings. But most of life isn't a closed system with enumerated rules and clear outcomes. If you try to explicitly name all the factors that make someone a good friend or co-founder, you might come up with some good indicators, but your crude map will also mislead you about the territory.

This is why "wisdom is knowledge that can't be transmitted." Wise people have navigational skill, not a long list of rules.

Sign up to be notified when my book, You Can Just Do Things, is available for purchase.

Ownership

THORSTEN BALL · 08 JUL 2026 · SOURCE

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It's only lightly edited.

I shared it before, but not here, because I didn't think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don't I? So here we are.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.



Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase "ownership" or allude to it, but haven't explained what "ownership" means to me in a while.

So, ownership.

If I ask you “can you own this?” or “can you take care of this?” or “are you on it?” — what I’m doing is I’m asking you to *own* it, to own the solution of a problem from end to end. From “we have a problem” to “we don’t have to think about it again.”

That means, when you say that you’re owning something, the expectation is that you…

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we’re actually trying to solve here. Maybe you think “the problem is that we need to migrate from using X to using Y”, but that’s not a problem, that’s a solution. The problem is likely something like “performance is bad”, “it’s not stable”, “it fails for customer x”. Maybe there’s other possible solutions to that? Think about those. What are the tradeoffs? What’s the best solution to go with considering these tradeoffs?

- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?

- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?

- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven’t confirmed yet?

- Think about how you’d test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?

- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!

- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say “here’s what I built, under these constraints, with these tradeoffs?”

- Test it manually. Yes, there’s automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?

- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it's actually deployed?

- If you think your colleagues needs to know about this change, because it's new feature they should all test, or it's a new convention in codebase, or maybe it's a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.

- Do customers need to know? Who reported the bug? Who's blocked? Let them know.

- Does the world need to know? Announce that it's out.

- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that's a lot. And there's actually more, because I'm sure I forgot some stuff.

But that's how you build a product in a small team. We don't have PMs, we don't have a Q&A department. We're small, but we're *great*, we can do all of that.

And it's always okay to ask for help, it's okay to ask questions, it's okay to redo things and triple-check. What's not okay is to implicitly assume that someone else will do the things here that you haven't thought about.



"How does this apply juniors? You can't expect them to really do all of that?"

I've been asked these questions, or variations of them, after I shared the thoughts above and here's my answer.

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, … — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.



You know what *you* should own? A subscription to this newsletter:

Zen of AI Coding

YOAV AVIRAM · 08 MAR 2026 · SOURCE

Dedicated to all those who are sceptical about the significance of agentic coding, and to those who are not, and are wondering what it means for the future of their profession. The title is an homage to Zen of Python by Tim Peters. Unlike Tim, I am not a zen master. My only aim is to take stock of where we are and where we might be heading. I have been building with coding agents daily for the past year, and I also help teams adopt them without losing reliability or security.

1. Software development is dead
2. Code is cheap
3. Refactoring easy
4. So is repaying technical debt

5. All bugs are shallow
6. Create tight feedback loops
7. Any stack is *your* stack
8. Agents are not just for coding
9. The context bottleneck is in your head
10. Build for a changing world
11. When considering Buy vs. Build, the answer, more often, is build
12. Fast rubbish is still rubbish
13. Software is a liability, a product is an asset
14. Moats are more expensive
15. Build for agents
16. Anticipate modes of failure

Software development is dead

You do not need to write another line of code if you do not want to. Coding agents can accomplish most coding tasks with the right direction.

Software development, as the act of manually producing code, is dying. A new discipline is being born. Your role in it is vital, but it is no longer centered on typing code. It is centered on framing problems, shaping context, defining constraints, and judging outcomes.

The marginal cost of code is collapsing. That single fact changes everything that follows.

Code is cheap

The economics of software have changed.

When coding is cheap, implementation stops being the constraint. You can build ten things in parallel. You cannot decide, validate, and ship ten things in parallel, at least not without changing the rest of the pipeline.

Cost of delay shifts. It is no longer about developer days. It is about time stuck in other bottlenecks: product decisions, unclear requirements, security review, user testing, release processes, and operational risk. Agents can flood these queues. Inventory grows. Lead time grows. Delay becomes more expensive, not less.

This changes prioritization. The highest leverage work is what unblocks shipping: tight feedback loops, tests, evals, guardrails, observability, and clear acceptance criteria. Anything that turns “we can build it” into “we can trust it”.

Agents can help alleviate some of these bottlenecks. The challenge is applying them outside of coding (see Agents are not just for coding)

Refactoring is easy

The cost of changing your mind is lower than it has ever been. Architectural decisions that once felt permanent are now provisional.

You chose React. Two months later, you regret it. Ask an agent to rewrite the project.

Making imperfect decisions is no longer fatal. In fact, it can be productive. A flawed reference implementation provides better context than a pristine specification. Agents reason more effectively from concrete artifacts than from abstract intent.

Rapid iteration is now the default mode. Over the last three months, we rebuilt our CMS four times from scratch, each with a different architecture. Each time we learned more about what we actually needed and what works.

When code is cheap, you can run more small bets.

So is repaying technical debt

There is little excuse for stale dependencies or ignored security patches. Updating libraries, migrating APIs, modernizing patterns. These are prompts away.

Prune your code regularly. Upgrade aggressively. Keep the surface clean.

Technical debt has not disappeared. But the cost of servicing it has dropped. That changes the incentive structure. Neglect becomes less defensible.

All bugs are shallow

Linus's law states that "given enough eyeballs, all bugs are shallow." Those eyeballs are now abundant.

Ask one model to review your code. Ask another. They may find different issues. Ask them to fix what they find. Often they will succeed.

How "dense" are bugs given a piece of code is a philosophical question. Bugs are not magically gone. Agents do not achieve perfection. From a practical perspective, however, the limiting factor is no longer the number of reviewers. It is the tightness of your feedback loops.

The claim here is profound: comprehension of the codebase at the function level is no longer necessary. At the same time, relying entirely on the models is a recipe for disaster. You still have a job! (see: create tight feedback loops, anticipate modes of failure)

Agents can sometimes one-shot a task.

We've built agents that create a complete custom-made marketing website in one shot, but this is by far the exception. In most cases, agents operate best when iterating towards a well-defined goal, guided by feedback.

Good tests are the first feedback loop. The agent will iterate until the tests pass. It is your responsibility to make sure the tests are adequate (by instructing the agent to create good ones) and to make sure the agent does not cheat (by weakening the tests or simply skipping them).

Give your agent access to your CI to create a second feedback loop. Give it access to your server logs to create a third. Giving the agent away to visually inspect a UI is yet another example.

Your job is to design environments where iteration converges toward correctness instead of drifting toward plausible nonsense.

Velocity without feedback is chaos.

Any stack is your stack

Agents are competent across most major stacks where training data exists. Since you are not writing the code, your attachment to a specific stack becomes less relevant.

Do not limit yourself to what you personally know. Your conceptual understanding transfers more than you think.

When you lack terminology or domain knowledge, ask the agent to brief you. The barrier to entry into unfamiliar ecosystems is lower than it has ever been.

Agents are not just for coding

Coding is only the beginning.

Agents can assist with business analysis, UX, infrastructure, operations, ad campaigns, analytics configuration, even accounting workflows.

I migrated four WordPress blogs from an overpriced host to Hetzner in minutes after postponing the task for years. The blocker was never a technical difficulty. It was attention. Claude completed the task in 15 minutes.

Agents reduce the activation energy required to execute tedious but valuable work.

Grant access carefully. Limit scope. Audit results. Direction without oversight is reckless.

The context bottleneck is in your head

Context engineering has improved. Tooling has improved.

Yet when running multiple agents in parallel, the real bottleneck is human coordination.

You must track what each agent is doing. You must remember which assumptions were made. You know what questions to ask next.

As agents become better at managing their own context, your ability to supervise them becomes the limiting factor.

The constraint is no longer tokens. It is cognition.

Build for a changing world

New models. New capabilities. New frameworks. New skills.

The ground shifts daily. Some models are better at reasoning. Some at coding. Some at translation.

Flexibility is not optional. It must be built into your workflow and products. Both must be engineered for experimentation and adaptability.

When considering Buy vs. Build, the answer, more often, is build

When code was expensive, buying made sense. When code approaches zero marginal cost, the calculus changes.

Every paid API is now a question. Could we replicate this internally? Should we?

For our QA agent, we needed full-page screenshots of live sites. Many APIs offered this. Instead, we deployed Playwright to AWS Lambda, tailored to our needs, and hardened.

This is not a blanket argument against SaaS. Maintenance, security, and scale still matter. But many small and medium services that once required vendors are now within reach.

Some call this the end of SaaS.

Fast rubbish is still rubbish

Speed is intoxicating. It is also dangerous.

You can generate enormous amounts of code quickly. But velocity and lines of code written were always terrible indicators of progress.

Refactoring is cheap, but it is cheaper to detect flawed direction early.

Discipline matters more, not less, when execution is frictionless.

Software is a liability, a product is an asset

Software costs money to maintain. It only becomes an asset when it produces value for someone.

This was always true. It is harder to remember now because building is so easy.

It is tricker then ever to resist the temptation to add features. Resist it. Build what is used. Kill what is not.

Feature gaps close quickly. What once took years to replicate can now take months. What took months can now take days.

A feature set is no longer a durable moat.

Moats shift toward distribution, data, brand, network effects, regulation, and ecosystem integration.

The barrier to entry falls. The barrier to defensibility rises.

There is a new hierarchy.

At the base are models (Opus 4.6, GPT-5.3-codex). On top of them sit agents (Claude Code, OpenClaw). On top of agents sit services.

Today, agents use services designed for humans. It works, but it is inefficient. The next step is services designed for agents (Moltbook being the harbinger).

Make your services accessible to agents. Expose structured context. Provide machine-readable surfaces.

Agentic commerce is emerging. In some cases, a markdown endpoint is enough. In others, the service itself must be designed agent-first.

Agent-first is not a tooling choice. It is a product decision. It changes your interfaces, your reliability model, your metrics, and what you build next.

We are building an agent-first CMS. If we succeed, humans will not need to use it, though they can.

AX is the new UX

Do not assume competence implies perfection.

Like an engineer overseeing the construction of a bridge, your job is not to lay bricks. It is to ensure the structure does not collapse.

Agents will make mistakes. They will drift off course. They will introduce subtle flaws. They will create security vulnerabilities. One of mine attempted to commit a .env file to git. They can be inventive in how they self-sabotage.

Be one step ahead. Ask yourself how this could break. Where could it leak. What could it expose. What assumptions is the agent quietly making.

Play the what-if game relentlessly. Then build guardrails.

Automate checks. Lock down permissions. Isolate environments. Add monitoring. Create recovery paths. Make rollback trivial.

Remember, code is cheap, and you have new superpowers. Build tools to allow you to probe deep, discover vulnerabilities, and test for anticipated failure modes. Design systems that expect failure and degrade gracefully.

This is the distinction between software development and software engineering.

I work with engineering and product teams adopting agents. The goal is simple: ship faster without lowering standards.

I usually help in two ways:

Agent-first software engineering
We turn agents into part of the system, not a novelty. We design workflows, guardrails, and supporting tools so the team can delegate confidently.

If you want to operationalize coding agents without turning your codebase into a casino, get in touch.

Agent-first product strategy
I help companies shift products from human-centric interfaces to agent-accessible, and where it makes sense, agents-first. That means clarifying what agents should be able to do, what constraints they must operate under, and what needs to change in your APIs, permissions, reliability model, and product surface area to support them.

This typically results in an agent-access roadmap, an operating model, and a prioritized set of product changes that make agents a first-class user.

I'm the co-founder of WhiteBoar, where AI agents build your brand and launch visually striking marketing websites, complete with professional multi-lingual content, in minutes, not months. Contact me at yoav@whiteboar.it.

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!